

FileFlux

In-Memory Unix Filesystem Emulator

Project Report — Object-Oriented Programming

College: Newton School of Technology

Repository: Silence91169/FileFlux (branch: main)

Date: April 2026

Team Members

- M1 — Abhishek Sharma(Lead / Architecture)
- M2 — Agrim Kumar Malhotra (Core Engine)
- M3 — Mohan Kumar CR(Nodes)
- M4 — Chaitanya Singh (Documentation)
- M5 — Satya Yadav (Testing & Frontend UI)

1. Problem Statement

Modern applications and learning platforms often require an isolated, safe, and lightweight environment to simulate file system operations without affecting the host OS. Existing solutions typically involve containerization (like Docker) or complex virtual machines, which are resource-heavy, slow to spin up, and overkill for simple tasks like teaching basic Unix commands or running constrained terminal-based applications in the browser.

FileFlux addresses the following critical gaps:

- **Heavy Setup overhead:** Setting up real filesystems for educational or ephemeral purposes is slow and error-prone.
- **Lack of safety:** Running commands on a host filesystem can lead to accidental data loss or security vulnerabilities.
- **Portability issues:** Sandboxed environments are difficult to port directly into a web browser without complex WebAssembly bindings.
- **No lightweight browser emulation:** Developers lack a simple, dependency-free, pure-JavaScript implementation of a Unix-like filesystem.

2. Proposed Solution

FileFlux is an elegant, purely in-memory Unix filesystem emulator that operates seamlessly in both Node.js backends and browser-based frontends. It provides:

- **Virtual Directory Tree:** A purely logical representation of files and folders using hierarchical Node structures.
- **Deterministic Path Resolution:** Accurate traversal through absolute (/), relative (./, ../), and deeply nested paths.
- **Wildcard Logic (*):** A unique deterministic wildcard resolution system that automatically resolves to the lexically smallest directory, enabling intelligent command completion and traversal.
- **Web-based Terminal UI:** A sleek, responsive frontend terminal that interacts directly with the in-memory FileSystem engine without requiring a backend server.
- **Object-Oriented Design:** Highly modular architecture built on robust

OOP principles allowing easy extension of node types and commands

3. Main Modules

Module	Description
Core Nodes (Nodes.ts)	Defines the abstract <code>FileSystemNode</code> and its concrete implementations: <code>FileNode</code> and <code>DirectoryNode</code> . Handles parent-child relationships and metadata.
File System Engine (FileSystem.ts)	Acts as the central orchestrator. Maintains the root directory and current working directory state. Provides standard Unix interfaces like <code>mkdir</code> , <code>cd</code> , <code>pwd</code> , and <code>ls</code> .
Path Resolver (PathResolver.ts)	Parses input paths. Iteratively navigates the virtual directory tree, handling <code>.</code> and <code>..</code> effectively to determine the target Node.
Wildcard Resolver (WildcardResolver.ts)	Handles the <code>*</code> logic by querying a directory's children, filtering directories, and selecting the lexically smallest directory to resolve ambiguous paths.
Frontend Terminal (app.js)	A vanilla JavaScript and CSS UI that provides a retro terminal interface. Captures user input, dispatches commands to the engine, and renders the current directory tree in real-time.

4. Technology Stack

Layer	Technology	Reason
Backend Engine	TypeScript / Node.js	Strong typing ensures reliable abstraction over node types. Pure TS implementation ensures there are zero native OS dependencies.

Frontend Engine	JavaScript (ES6)	Directly ported from the TS backend (fs.js) to run independently in the browser without any bundlers.
UI / Presentation	Vanilla HTML, CSS, JS	Keeps the application lightweight, fast, and dependency-free. Allows direct DOM manipulation for the terminal and directory tree renderer.

5. OOP Concepts Used

FileFlux is fundamentally driven by Object-Oriented Programming. The core engine models the file system as an interconnected graph of objects.

a) Abstraction

The system defines a unified interface for all filesystem elements through the `FileSystemNode` class. The internal complexities of how a node stores data or children are hidden from the `PathResolver` and `FileSystem`.

b) Inheritance

Both `FileNode` and `DirectoryNode` inherit from the base `FileSystemNode`. This allows them to share common attributes like name, parent reference, and creation timestamp.

```
class FileSystemNode {
    constructor(name, parent = null) { ... }
    getName() { return this._name; }
    getPath() { ... }
}

class DirectoryNode extends FileSystemNode {
    // Inherits name/path, adds child management
}
```

c) Polymorphism

The `isDirectory()` method is defined as abstract in the base class and implemented differently in subclasses. The `PathResolver` simply calls `current.isDirectory()` without needing to know the exact subclass type.

```
class FileNode extends FileSystemNode {
    isDirectory() { return false; }
}

class DirectoryNode extends FileSystemNode {
```

```
isDirectory() { return true; }  
}
```

d) Encapsulation

Nodes encapsulate their internal state (e.g., the `_children` Map in `DirectoryNode` or `_content` string in `FileNode`). Access is only permitted through public methods like `addChild()`, `getChild()`, and `read()`.

6. Design Patterns Used

Composite Pattern

What is it? A structural pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

Where used? The `DirectoryNode` and `FileNode` structure. A directory can contain files and other directories. The `FileSystem` interacts with the root `DirectoryNode`, treating the entire hierarchy uniformly.

Dependency Injection

Where used? The `PathResolver` takes an instance of `WildcardResolver` in its constructor. This decouples path parsing from the complex logic of resolving `*`, making it easier to mock or change wildcard behavior without modifying the path resolver.

7. System Architecture

FileFlux uses a stateless logic flow for resolving commands. For example, when `cd foo/*/bar` is executed:

1. The command is dispatched from `app.js` to `FileSystem.cd()`.
2. The `PathResolver` splits the path into components: `['foo', '*', 'bar']`.
3. It navigates to `foo` (getting the `DirectoryNode`).
4. It encounters `*`, calls `WildcardResolver.resolve(foo_node, '*')`, which finds the lexically first directory inside `foo`.
5. It navigates into that matched directory, then attempts to find the child `bar`.
6. If successful, `FileSystem.cwd` updates to the `bar` `DirectoryNode`.

8. Project Structure

```
FileFlux/
├── backend/
│   ├── src/
│   │   ├── core/
│   │   │   └── Nodes.ts           # Abstract and
concrete node classes
│   │   └── engine/
│   │       ├── FileSystem.ts      # Main FS state and
commands
│   │       ├── PathResolver.ts    # Tree traversal
logic
│   │       └── WildcardResolver.ts # '*' deterministic
resolution
├── frontend/
│   ├── app.js                    # Terminal DOM
manipulation and command mapping
│   └── fs.js                     # Browser-
compatible port of the backend engine
│   ├── index.html               # UI layout
│   └── style.css                 # Terminal
aesthetics
└── README.md
```

9. Getting Started

Because FileFlux includes a standalone frontend, you don't need a backend server to run the emulator.

1. Clone the repository.
2. Open `frontend/index.html` in any modern web browser.
3. Use the terminal interface to interact with the dummy seeded filesystem.
Available commands: `mkdir`, `cd`, `pwd`, `ls`, `clear`, `help`.